

★ YouTube → Faisal Memon

Exception Handling → The exception handling in Java is one of the powerful mechanism to handle the runtime errors so that normal flow of the application can be maintained.

★ What is exception → In Java, exception is an event that disrupts the normal flow of the program. It is an object which is thrown at runtime.

○ → Advantage of Exception Handling:

The core advantage of exception handling is to maintain the normal flow of the application.

★ Type of Exception

There are mainly two type of exception; checked and unchecked where 'error' is considered as unchecked exception.

• The Sun microsystem says there are three type of exceptions.

- ① CheckedException
- ② UncheckedException
- ③ error.

★ Difference between checked and unchecked exceptions

① Checked Exception: The classes that extend

Throwable class except RuntimeException and Error are known as checked exceptions
E.g → SQLException, IOException,
checked exceptions are checked at compile-time

② → Unchecked Exception: The classes that extend RuntimeException are known as unchecked exceptions. e.g → ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException etc. Unchecked exceptions are not checked at compile-time rather they are checked at runtime

- ③ Error : → Error is irrecoverable
e.g → Out of Memory Error. Virtual Machine Error.
Assertion Error etc.

Checked and Unchecked Exception

Checked Exceptions	Unchecked Exceptions
• Exception which are checked at Compile time called Checked Exception. If a method throws a checked exception, then the method must either handle the exception or it must specify the exception using throws keyword. Examples:- - IOException - SQLException - DataAccessException - ClassNotFoundException - InvocationTargetException - MalformedURLException	• Exceptions whose handling is NOT Verified during compile time. • These exceptions are handled at run time → e.g by JVM after they occurred by using the try and catch block. • <u>Examples:</u> • NullPointerException • ArrayIndexOutOfBoundsException • IllegalArgumentException • IllegalStateException

★ Java try block

Java try block is used to enclose the code that might throw an exception. It must be used within the method.

"Java try block must be followed by either catch or finally block."

① Syntax of java try-catch

```
try {  
    // code that may throw exception  
} catch (Exception_class_Name ref) { }
```

② Syntax of try-finally block

```
try {  
    // code that may throw exception  
} finally { }
```

★ Java catch block

Java catch block is used to handle the Exception. It must be used after the try block only. You can use multiple catch block with a single try.

Problem without exception handling

Let's try to understand the problem if we do not use try-catch block.

Eg. →

```
Public class Testtrycatch1 {  
    · public static void main(String args[]) {  
        int data = 50/0; // may throw exception  
        System.out.println("rest of the code...");  
    }  
}
```

Output: Exception in thread main Java.lang.
ArithmeticException: / by zero

As displayed in the above example, rest of the code is not executed.

* These can be 100 lines of code after exception. So all the code after exception will not be executed.

* Solution by exception handling:

Let's see the solution of above problem by Java try-catch block.

E.g-2:

```
Public class Testtrycatch2 {  
    public static void main (String [] args) {  
        try {  
            int data = 50/0;  
        } catch (ArithmeticException e) {  
            System.out.println(e);  
            System.out.println("rest of the code...")  
        }  
    }  
}
```

Output: Exception in thread main: Java.lang.
ArithmeticException : / by zero.
rest of the code...

"Now, as displayed:"

★ Java Multi catch block:

If you have to perform different tasks
at the occurrence of different Exceptions,
use java multi-catch block.

e.g.:

```
public class TestMultipleCatchBlock {  
    public static void main (String [] args) {  
        try {  
            int a[] = new int [5];  
            a[5] = 30/0;  
        }  
        catch (ArithmeticException) {  
            System.out.println("Task 1 is completed");  
        }  
        catch (ArrayIndexOutOfBoundsException) {  
            System.out.println("Task 2 completed");  
        }  
        catch (Exception e) {  
            System.out.println("Common task");  
        }  
        System.out.println("rest of the code");  
    }  
}
```

Output:

Task 1 completed
rest of the code...

★ Java nested try examples

Let's see a simple example of Java nested try block.

Eg →

```
class Except6 {  
    public static void main (String [] args) {  
        try {  
            try {  
                System.out.println ("going to divide");  
                int b = 39/0;  
            } catch (ArithmeticException e) {  
                System.out.println (e);  
            }  
            try {  
                int a [] = new int [5];  
                a[5] = 4;  
            } catch (ArrayIndexOutOfBoundsException e) {  
                System.out.println (e);  
            }  
            System.out.println ("other statement");  
        } catch (Exception e) {  
            System.out.println ("handeled");  
        }  
        System.out.println ("normal flow");  
    }  
}
```


★ Java finally block:

Java finally block is a block that is used to execute important code such as closing connection, stream etc.

• Java finally block is always executed whether exception is handled or not."

Java finally block follows try or catch block:-

★ Usage of Java finally

Let's see the Java finally example where exception does not occur.

```
→ class TestFinallyBlock {  
    public static void main(String[] args) {  
        try {  
            int data = 25/5;  
            System.out.println(data);  
        }  
        catch (NullPointerException e) {  
            System.out.println(e);  
        }  
        finally {  
            System.out.println("finally block is
```

```

        always executed");
    }
    System.out.println("rest of the code ...");
}
}

```

Output: "finally block is always executed
rest of the code..."

★ Java throw keyword:

The Java throw keyword is used to explicitly throw an exception.

"We can throw either checked or unchecked exception in Java by throw keyword."

The throw keyword is mainly used to throw custom exception.

E.g:

```

public class TestThrow1 {
    static void validate(int age) {
        if (age < 18)
            throw new ArithmeticException("not valid");
        else
            System.out.println("welcome to vate");
    }
}

```



```

Public Static void main (String [] args) {
    Vali date (13);
    System.out.println("rest of the code...");
}
}

```

Output: Exception in thread main java. lang
ArithmeticException: not valid.

★ Java throws keyword:

The Java throws keyword is used to declare an exception. It gives an information to the programmer that there may occur an exception so it is better for the programmer to provide the exception handling code so that normal flow can be maintained.

★ Syntax of Java throws

```

return_type method_name ( ) throw exception_
class_name {
    // method
}

```



Java throws Example

```
import java.io.IOException;
```

```
class Testthrows {
```

```
    void m () throws IOException {
```

```
        throw new IOException("device error");  
        // checked exception
```

```
    }
```

```
    void n () throws IOException {
```

```
        m();
```

```
    }
```

```
    void p () {
```

```
        try {
```

```
            n();
```

```
        } catch (Exception e) {
```

```
            System.out.println("exception handled");
```

```
        }
```

```
    }
```

```
    public static void main (String [] args) {
```

```
        Testthrows t Obj = new Testthrows();
```

```
        Obj.p ();
```

```
        System.out.println("normal flow;..");
```

```
    }
```

```
}
```


Output:

exception handled
normal flow ...

★ Java Custom Exception

If you are creating your own Exception that is known as custom exception or user-defined exception. Java custom exceptions are used to customize the exception according to user need.

By the help of custom exception, you can have your own exception and message.

Example — "In screenshot → github repo"

• If you want to take the class-based example then follow → the GitHub repository •

* Java - Exception Interview Question:

- ① What is the difference between Checked and Unchecked
- ② What is the difference between throw and throws?
- ③ What happen if an exception is not handled in Java?
- ④ Can we have multiple catch blocks in Java?
- ⑤ What is the difference between final, finally and finalize()?

Figure: Exception Hierarchy in Java

BenchResources.Net

